

Topics of mongodb :

- What is MongoDB?
- Environment Setup
- MongoDB Compass
- VS Code setup
- Insert Documents into collection
- Data Types
- Find, Project, Sort and Limit functions
- Update functions
- Misc. things
- Upsert function
- Delete function
- Comparison Operators
- Logical Operators
- Array functions
- Aggregation Pipelines
- Schema Validation
- Lookup (joins)
- Indexing
- MongoDB Atlas

INTRODUCTION TO MONGODB

MongoDB :

- It is a NoSQL database.
- It can store a huge amount of data.
- It is extremely popular and used by top companies.
- Data in MongoDB is stored in BSON (Binary JSON). This is stored as binary values and is faster and more efficient than standard JSON (JavaScript Object Notation).

How MongoDB differs from SQL:

- **SQL:** Data is structured into tables with rows and columns.
- **MongoDB:** Data is organized into documents and collections.

Example (RDBMS table):

Name	Age
Dinesh	20
Arjun	18

In SQL, the records ("Dinesh", 20; "Arjun", 18) are stored as rows and columns in a fixed schema table.

MongoDB Equivalent:

```
json
```

```
{
  "Name": "Dinesh",
  "Age": 20
}
{
  "Name": "Arjun",
  "Age": 18
}
```

In MongoDB:

- Rows are called **documents**
- Columns are called **fields**

- Tables are called **collections**
- Data is stored without a fixed schema (NoSQL)

Cons of MongoDB:

- No normalization needed (does not replicate data using foreign key links), unlike traditional SQL databases.
- Flexible schema (no fixed structure required).

How MongoDB stores large amounts of data:

- **Sharding:** Used to distribute large datasets across multiple servers. Each split is called a shard, and each server stores a shard.
- **Replication:** Creates copies of the data in those servers. This is beneficial for fault tolerance (protects against bugs, crashes, or server problems).

ENVIRONMENT SETUP FOR MONGODB

1. **Download the community server of MongoDB :**
<https://www.mongodb.com/try/download/community>
2. **Download the shell of MongoDB :**
<https://www.mongodb.com/try/download/shell>

The MongoDB Community Server is the main file because it process everything. After downloaded install it and

.)click next and click continue to install every files and another option is
.)custom it installs only as per the req of the user

The MongoDB compass is used to create new db's servers's and everything related to MongoDB(Data,Server,Database,Querie's and etc...)

In the system file :

C:\Program Files\MongoDB\Server\8.2\mongosh-2.5.9-win32-x64\mongosh-2.5.9-win32-x64\bin

→ copy the path from navigation path

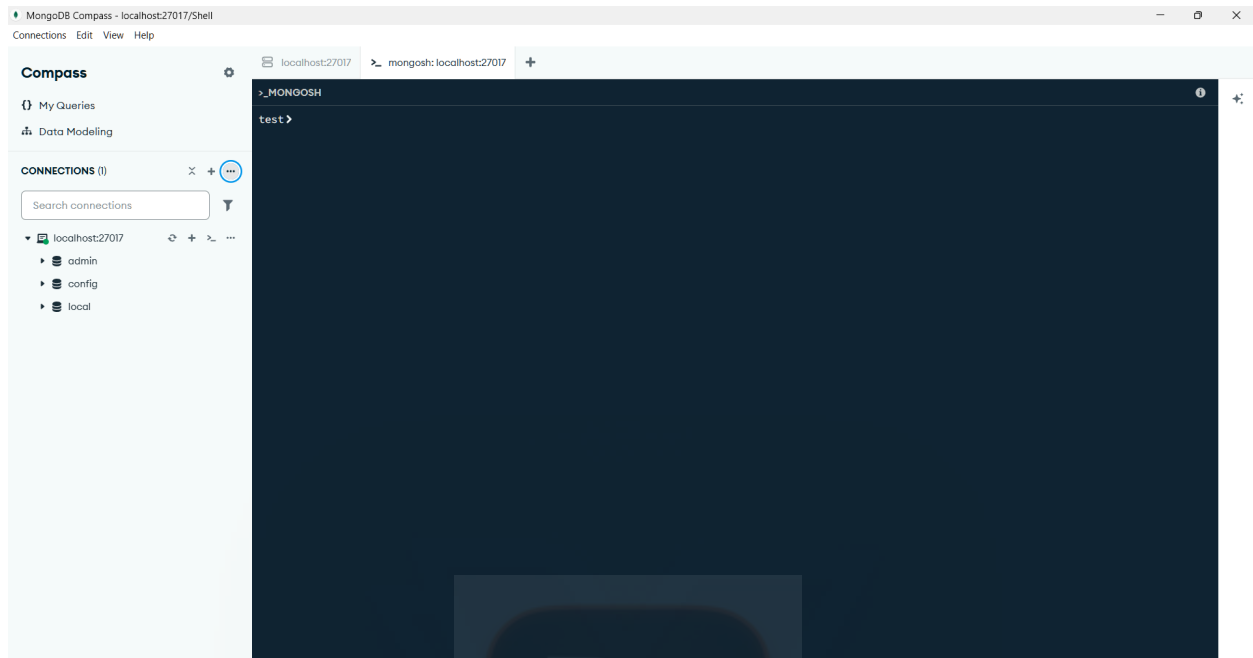
→ go to environment variables select path and add this as new path and add it

→ after this enter "mongosh" in terminal and if it comes true>> then u setupped the mongoshell correctly

MONGODB COMPASS

Step 1 : Add New Connection :

In this dont make any changes just click save and connect



In the above image the

Localhost:27010 is our server (DataServer)

Admin , config , local are the database

Here Are the Steps and create our own database

Step 1 : creating our own DataBase

Use Mychoice //here Use is query and Mychoice is the DataBase

Switched to db Mychoice

Step 2 : Displaying our DataBase in Shell

Show dbs //here show dbs is query and Mychoice is the newly added DataBase

Admin

config

local

Mychoice

Step 3 : creating collection to display our DB in the server list

db.createcollection("T-shirts") //here createcollection is function and T-shirts is the collection inside the Database Mychoice

```
{ok : 1} //here this is the json format tells an collection is created
```

Step 4 : switch to other database

Use admin

Switched to db admin //here the db is changed(from mychoice to admin)

Step 5 : Deleting the collection and also DataBase

clothes>db.t-shirt.drop() //drop() is the function and it will clears out the created collection the Mychoice Database

[clothes.db](#).dropDatabase() //dropdatabase is an function and it entirely deletes the database

In vscode setup just go to extension and type “mongodb for vscode” and add connection

Insert Document into collections

First we are going to insert the document (row) into the collection(table)

Localhost:27010 —> server

- - admin —> database

- - config —>database

- - hospital —>database

- - department —>collection(table)

- - local —>database

i] Inserting one document(row)

Db.department.insertOne — here db is database and department is collection and insertone is used to insert only one document-data(row)

```
db.department.insertOne( { departmentname : “nephrology” , totaldoctors : 5 } )
```

ii] Inserting many documents(row)

Db.department.insertMany([{ , } , { , }]) — here db is database and department is collection and insertMany is used to insert many document-data(row) in an list of array

```
db.department.insertMany( { departmentname : "nephrology" , totaldoctors : 5 },
departmentname : "urology" , totaldoctors : 10 }, departmentname : "neurology" , totaldoctors :
16 } )
```

iii] to see the data in an particular collection

db.department.find() — here db is database and department is collection and find is used to see the data in the collection(table)

```
db.department.find()
```

DataTypes

The database has the different kind of values that are declared to an variables

```
{
  Departmentname : "pulmonology",    // String
  Totaldoctors : 21,                 // Integer (Number)
  score : 9.0,                       // Double (Number)
  Diagnosed : true,                  // Boolean
  Date : new Date( )                 //It displays the date and time( in utc form)
  Doctors : [                        // Array
    "Dinesh",
    "Reena"
  ],
  Related departments : {            // Embedded / Nested Document
    Department name : "respiratory medicine",
    Totaldoctors : 16
  }
}
```

“MongoDB stores data in BSON → which supports String, Number, Double, Boolean, Array, and even nested documents inside documents.”

So these above are the possible datatypes of MongoDB

Find, Project, sort and limit functions

i) inside the find function we can pass the argument with the declared value of an document

Inside the find() with an filter we can pass a condition to match specific documents.

```
db.Department.find({ departmentname: "pulmonology" })
```

then the details of the particular document will show

```
{
  _id: ObjectId('69240052369fecb31dbf3f4a'),
  departmentname: 'pulmonology',
  totaldoctors: 21,
  score: 9,
  diagnosed: true,
  date: 2025-11-24T06:50:58.690Z,
  doctors: [
    'Dinesh',
    'Reena'
  ],
  Related departments: {
    departmentname: 'respiratory medicine',
    totaldoctors: 16
  }
}
```

ii) project or projection

And also we can pass another thing in the find function which is called as projection or Projection is used to select which fields to show or hide.

```
db.Department.find({ departmentname: "pulmonology" },
  {departmentname : true , totaldoctors : true})
```

either we can project in the mongosh or gui in mongodb compass

iii) sorting

the sorting is used to display the value in ascending to descending(low to high) or descending to ascending(high to low)

- **-1 for descending (high to low), 1 for ascending (low to high).**

```
db.Department.find().sort({totaldoctors : -1}) //displays the value form high to low
```

using projection in the sort inside the find() -- displays only the document or data we want to see

iv) limit()

displays only the needed value limit(2) or used to show only a specific number of documents.

```
db.Department.find( { , {departmentname : true , _id : false} }.sort({totaldoctors : -1}).limit(2)
```

//displays only the top 2 data/document

🔥 Quick Memory Hack

Find → what to search

Project → what to show

Sort → in what order

Limit → how many results

- `find()` = Search 🔍
- `projection` = Filter what you want to see 🎯
- `sort()` = Arrange the results 📊
- `limit()` = Cut off after N results ✂️

Update functions

The update function is used to update an argument

I. 📝 updateOne

The updateone is used to update the value in an document

E.g. → `db.Department.updateOne( {departmentname : "UROLOGY" }, {$set:{departmentname : "Urology"}})`

If the multiple documents have same documents then we can use the `_id` value to modify it

```
db.Department.updateOne( { _id: ObjectId('6923f0bbc4148020b79cf71e') } , {$set:{
departmentname:"Urologist"}})
```

Here the

Db – > database

Department – is the collection(table)

updateone() – function is used to change an documents value

```
( {departmentname : "UROLOGY" }, {$set:{departmentname : "Urology"}})
```

```
( { _id: ObjectId('6923f0bbc4148020b79cf71e') } {$set:{ departmentname:"Urologist"}}
{$set:{departmentname : "Urology"}} )
```

– from “UROLOGY” TO “Urology” it will change in e.g.1 using id value the corresponding value change to “urologist”

i) Updating an value

```
db.Department.updateOne( {departmentname : "UROLOGY" }, {$set:{departmentname : "Urology"}})
```

ii)Removing an value

```
db.Department.updateOne( { _id: ObjectId('6923eecd73865c5836acb940') },
{$unset:{departmentname : ""}})
```

2. updateMany

It will update an multiple documents at same time

i) adding an value or updating an value

```
db.Department.updateMany( {diagnosed:{$exists:false}} , {$set:{diagnosed:true}})
```

Here the collections does not have the “diagnosed” field so using the set operator we can add the field in each document

ii)removing an value

```
db.Department.updateMany( {diagnosed:{$exists:true}} , {$unset:{diagnosed:""}})
```

Here the “diagnosed” field will be removed from the each and every documents

Keywords :

1.set :

The set keyword is an attribute or add operator which is used to add an field or create an field **syntax** `{$set:{ condition } }`

```
{ $set: { fieldName: newValue } }
```

2.unset :

The unset keyword is an attribute which is used to remove an field or remove the key : value pair in an document **syntax** `{$unset:{ condition } }`

```
{ $unset: { fieldName: "" } }
```

3.exists :

The exists keyword is an attribute or which is used to check whether an key or field is exist in the document **syntax** `{$exists: key }`

```
{ field: { $exists: true } } // field present
{ field: { $exists: false } } // field not present
```

✓ 1. What is a DOCUMENT?*

A **document** is one complete record inside a collection.
So **this entire block** is **one document**.

```
...
{
  _id: ...,
  departmentname: ...,
  totaldoctors: ...,
  diagnosed: ...
}
```

✓ **This whole thing = Document**

✓ 2. What are FIELDS?*

Fields are the **keys** inside a document.

Your fields:

```
* `_id`
* `departmentname`
* `totaldoctors`
* `diagnosed`
```

These are also called **field names**.

✓ 3. What are VALUES?*

Values are things **assigned to fields**:

```
* `_id` → `ObjectId('6923f0bbc4148020b79cf71e')`
* `departmentname` → `'Urologist'`
* `totaldoctors` → `4`
* `diagnosed` → `false`
```

✓ 4. What are ATTRIBUTES?*

In MongoDB, **attribute = field**

So:

```
departmentname = attribute
totaldoctors   = attribute
diagnosed      = attribute
```

They mean the same thing.

✓ 5. What are KEYWORDS? **

Keywords are ****MongoDB operators**** like:

```
* ` $set `
* ` $unset `
* ` $exists `
* ` $inc `
* ` $push `
```

In update queries, keyword = operator.

Miscellaneous Things

i) Find and Findone() function

`db.department.find()` – this will show the entire documents of the collection

`db.document.find( { departmentname: 'Nephrology' } )` – this will fetch the documents and shows which documents are matching the field

```
{
  _id: ObjectId('6923eecd73865c5836acb940'),
  totaldoctors: 19,
  departmentname: 'Nephrology'
}
```

```
{
  _id: ObjectId('6923e89373865c5836acb93f'),
  departmentname: 'Nephrology',
  totaldoctors: 43
}
```

db.document.findOne({ departmentname: 'Nephrology' }) – this will fetch only one document which shows at top

Using limit() we can get the matching record but it will display the matching records and we have to access it (2 steps to be coded to access the field). But using the findOne() we can fetch only one document

```
{
  _id: ObjectId('6923eecd73865c5836acb940'),
  totaldoctors: 19,
  departmentname: 'Nephrology'
}
```

2.projection

It will pass field in the find() which is used to hide or show an field

db.document.find({ departmentname: 'Nephrology' } , { _id : false }) // { _id : 0 }

The above example will show only the departmentname and hides the id value

```
{
  departmentname: 'Nephrology',
  totaldoctors: 43
}
```

db.document.find({ departmentname: 'Nephrology' } , { _id : true }) // { _id : 1 }

The above example will hides the departmentname and also show the id value

```
{
  _id: ObjectId('6923e89373865c5836acb93f')
}
```

3.updateOne and updateMany() :

We can increment the value without using the set and unset keywords.

db.Department.updateMany({departmentname: "Nephrology"} , {\$inc:{totaldoctors : 12}})

It will increment the value as :

Pre value 10 and increment value : 12 so the new value will be 22

The Increment operator is used to increase and decrease a numeric field

```
db.Department.updateMany( { departmentname: "Nephrology" } ,
{ $ inc : { totaldoctors : 12 } } ) – prev value : 10 it becomes => 22
{ $ inc : { totaldoctors : -12 } } ) – prev value : 22 it becomes => 10
```

upsert() function :

Upsert = update + insert

If the doc does not exist - > update it
if does not exist - > insert an document

```
db.Department.updateOne( {departmentname : "dermatologist" } ,
                        { $set: {totaldoctors : 15 , score : 9} },
                        {upsert : true} )
```

In this you can update or change any value or else you can insert any value.

```
{
  _id: ObjectId('6927d8ca3e2db3be84980c22'),
  departmentname: 'dermatologist',
  score: 9,
  totaldoctors: 15
}
```

Delete() Function

You can delete an field

i) deleteOne() :

Deletes only one matching document

```
db.Department.deleteOne({ name : "dinesh" })
```

In deleteOne : mongodb deletes the first document if finds that matches the filter(top)

- > if 2 are same → one gets deleted
- > the other remains exactly same

This db.Department.deleteOne({ name : "dinesh" }) will deletes the only one matching field not all

ii) deleteMany() :

It will delete all the matching document value

This db.Department.deleteMany({ name : "dinesh" }) **will delete all the matching field..**

Comparison Operator

.) Comparison operators are special operators in MongoDB that are used inside queries to compare the values of fields with specific conditions.

.) Comparison operators compare a field's value with another value and return the documents that satisfy that condition.

.) comparison operators are the "conditions"

1.Greater then (gt)

db.Department.find({ totaldoctors : { \$gt : 10 } }) – this will return the document values of the field "**totaldoctors**" greater than 10

2.lessor then (lt)

db.Department.find({ totaldoctors : { \$lt : 20 } }) – this will return the document values of the field "**totaldoctors**" lesser than 20

3.lessor then or equal to (lte)

db.Department.find({ totaldoctors : { \$lte : 19 } }) – this will return the document values of the field "**totaldoctors**" lesser than 20(fetches 19 and lesser value then it)

4.greater then or equal to (gte)

db.Department.find({ totaldoctors : { \$gte : 19 } }) – this will return the document values of the field "**totaldoctors**" greater than or equal to 19(fetches 19 and greater value then it)

5.equal to (eq)

db.Department.find({ totaldoctors : { \$eq : 19 } }) – this will return the document values of the field "**totaldoctors**" equal to 19(fetches the field which is exactly equal to value 19)

6. not equal to (ne)

`db.Department.find( { totaldoctors : { $ne : 10 } } )` – this will return the document values of the field “**totaldoctors**” not equal to 10 (it fetches all the field and display the other entire documents with the field value)

7. in operator (in)

`db.Department.find( { totaldoctors : { $in : [ 19 , 10 ] } } )` – this will return the document values of the field “**totaldoctors**” which are 19 , 10 (it will only fetches the value declared inside the array)

8. Not in operator (nin)

`db.Department.find( { totaldoctors : { $nin : [ 19 , 10 ] } } )` – this will return the document values of the field “**totaldoctors**” except the field values 19 and 10 (it will fetches all the documents filed value except the value 10 and 19 which are declared inside the array)

Logical Operators

.) Logical operators are used to combine multiple conditions in a query.

.) If comparison operators are the “conditions,” logical operators are the “decision makers.”

MongoDB provides 4 main logical operators:

```

✓ $and          syntax :
✓ $or           db.collection.find({
✓ $not          $and: [ condition1, condition2 ]
✓ $nor          })

```

1. And Operator

Both the Conditions has to be satisfied

`db.Department.find( { $and : [ { departmentname : "dermatologist" } , { totaldoctors : 19 } ] } )` → this will return document if both the conditions (“**departmentname**”, “**totaldoctors**”) matches the field values in the document

2.Or Operator

At least one condition must be satisfied

db.Department.find({ \$or : [{ departmentname : "Urologist" } , { totaldoctors : 10 }] })

—> this will return the document at least one of the conditions declared inside the array (“**departmentname**”, “**totaldoctors**”) in which at least one value matches the field values in the document

3.Nor Operator

None of the condition must be True

db.Department.find({ \$nor : [{ departmentname : "Urologist" } , { totaldoctors : 10 }] })

) —> this will return the document except the conditions declared inside the array (“**departmentname**”, “**totaldoctors**”) in which these declared conditions will be omitted and will display's the other documents

4.Not Operator

Opposite of the condition

db.Department.find({ departmentmane : { \$not: { \$eq: "Nephrology" } } })

—> 🖱️ **Returns documents where departmentname is **not "Nephrology"**.

this will return the documents except the condition declared inside the array (“**departmentname**”) **\$not** is applied inside a field condition, not directly with multiple fields.

Operator	Meaning	Example
\$and	ALL conditions must match	two conditions true
\$or	ANY one condition matches	At least one true
\$nor	NONE should match	opposite of OR
\$not	NEGATE condition	must use eq, gt, lt, regex

Array functions

Array operators are used to work with array fields inside a document.

Example of a document with an array field :

```
{
  _id: 1,
  departmentname: "Urology",
  doctors: ["Ram", "Siva", "John"],
  totaldoctors: 3
}
```

Here, doctors is an array.

MongoDB gives us special operators to query or update array values.

1. \$in – Match array elements

Checks if any element inside an array matches a value.

```
db.Department.find({
  doctors: { $in: ["Ram"] }
})
```

✓ Returns document if “Ram” is inside the doctors array.

2. \$all – Match ALL given elements

Checks if an array contains every value you specify.

```
db.Department.find({
  doctors: { $all: ["Ram", "John"] }
})
```

✓ Returns document only if both Ram AND John are present.

3. \$size – Match array length

Find documents where array has exact number of elements.

```
db.Department.find({
  doctors: { $size: 3 }
})
```

```
}}
```

✓ Returns documents where the array length = 3.

4. \$elemMatch – Match element inside array with conditions

Useful when each element is an object, not just strings.

```
db.Department.find({  
  doctors: {  
    $elemMatch: { age: { $gt: 30 } }  
  }  
})
```

✓ Returns documents where at least one doctor has age > 30.

5. \$push – Add 1 element to array

```
db.Department.updateOne( { departmentname: "Urology" }, { $push: { doctors:  
"Karan" } } )
```

✓ Adds "Karan" to doctors array.

6. \$pop – Remove first/last element

```
db.Department.updateOne(  
  { departmentname: "Urology" },  
  { $pop: { doctors: 1 } } // removes LAST element  
)
```

```
db.Department.updateOne(  
  { departmentname: "Urology" },  
  { $pop: { doctors: -1 } } // removes FIRST element  
)
```

7. \$pull – Remove matching element

```
db.Department.updateOne(  
  { departmentname: "Urology" },
```

```
{ $pull: { doctors: "Ram" } }
)
```

✓ Removes only "Ram" from the array.

Array Operators :

- \$in → checks if array contains any of the values
- \$all → array must contain all values
- \$size → matches array length
- \$elemMatch → match object inside array
- \$push → add element to array
- \$pop → remove first/last element
- \$pull → remove matching element

Aggregation Pipelines

Aggregation Pipeline :

A step-by-step processing system that transforms your documents into whatever result you want.

Syntax : [{ step1 }, { step2 }, { step3 }]

Each step is an operator like:

\$match , \$group , \$project , \$sort , \$limit , \$count

Basic structure :

```
db.Purchase.aggregate([
  { stage1 },
  { stage2 },
  { stage3 }
])
```

① \$match → filter documents (like find)

```
db.Department.aggregate([
  { $match: { departmentname: "Nephrology" } }
])
```

② \$project → show/hide fields OR create new fields

```
db.Department.aggregate([
  { $project: { departmentname: 1, totaldoctors: 1 } }
])
```

③ \$group → do calculations (sum, count, average)

```
db.Department.aggregate([
  { $group: { _id: null, totalDoctors: { $sum: "$totaldoctors" } } }
])
```

Group by department:

```
db.Department.aggregate([
  { $group: { _id: "$departmentname", total: { $sum: "$totaldoctors" } } }
])
```

④ \$sort → sort documents

```
db.Department.aggregate([
  { $sort: { totaldoctors: -1 } } // descending
])
```

⑤ \$limit → limit output

```
db.Department.aggregate([
  { $limit: 2 }
])
```

⑥ \$count → count matching docs

```
db.Department.aggregate([
  { $match: { totaldoctors: { $gt: 10 } } },
```

```
{ $count: "BiggerDepartments" }  
])
```

When do we use Aggregation?

Whenever you want:

.)Calculations (sum, avg, min, max) .)Reports .)Counting .)Grouping .)Sorting
.)Showing/hiding fields .)Creating new fields .)Advanced filtering

It's basically MongoDB's data analytics engine.

Schema Validation

♦ 1. Schema Validation — What it actually means

"Database store's particular attributes and datatypes, each value has its own datatype, etc."

→ Correct — but MongoDB does this only if *you configure validation*.

MongoDB is *schema-less by default*, but **you can enforce rules** using validators.

✓ **Schema Validation = Rules for what kind of data the collection will accept**

You can enforce things like:

- datatype restrictions (**string, number, array, objectId**, etc.)
- required fields
- value ranges
- pattern matching (emails, phone numbers)
- nested structure rules

```
db.createCollection("patients", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age", "doctorId"],
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0 },
        doctorId: { bsonType: "objectId" }
      }
    }
  }
})
```

- patients → collection
- \$jsonSchema → rules for the shape of each document
- properties → list of fields + their allowed types
- required → must-have fields

This means:

- MongoDB **won't allow wrong data**
- If **doctorId** is not an ObjectId → insert fails
- If age is -5 → fails

✨ So your explanation is correct, just think of it as:

"Validation is like setting rules so only correct-shaped data enters the collection."

Lookup(joins)

♦ 2. \$lookup — Joining Two Collections

Your explanation:

"One document's ObjectId points to another document and lookup joins them."

→ Yup, that's the heart of **\$lookup**.

✓ **\$lookup = MongoDB's way of doing joins (like SQL join)**

It connects data from two collections using a matching field.

Example:

Doctors collection:

```
{ _id: ObjectId("abc"), name: "Kumar", deptId: 101 }
```

Department collection:

```
{ deptId: 101, departmentname: "Nephrology" }
```

Join them:

```
db.doctors.aggregate([
  {
    $lookup: {
      from: "Department",
      localField: "deptId",
      foreignField: "deptId",
      as: "departmentInfo"
    }
  }
])
```

Result:

```
{
  name: "Kumar",
  deptId: 101,
  departmentInfo: [
    { deptId: 101, departmentname: "Nephrology" }
  ]
}
```

- from → collection you pull data from
- localField → key in doctors
- foreignField → key in Department
- as → new field name to store joined documents

✨ Think of **\$lookup** as:

“Search in another collection and attach matching documents here.”

◆ Difference in one line

Schema Validation	Lookup Join
Controls what kind of data can be stored	Combines related data from two collections
Prevents wrong/invalid data	Works like SQL joins
Happens at insertion/update time	Happens at query/aggregation time

Index

✓ What is an Index?

An index in MongoDB is like the **index page of a book**.
Instead of checking every page, MongoDB can jump directly to the correct place.

✓ Why we use Index?

- Makes **searching faster**
- Makes **sorting faster**
- MongoDB avoids **scanning all documents**

✓ How to Create an Index

Example:

```
db.Users.createIndex({ name: 1 })
```

Here:

- **name** → field to be indexed
- **1** → ascending order

Now MongoDB creates a **sorted list of all "name" values**.

How it helps? (Your example)

Suppose collection has 100 names:

Arun

Bala

Charan

Dinesh

... up to 100 names

You create:

```
db.Users.createIndex({ name: 1 })
```

Now when you run:

```
db.Users.find({ name: "Arun" })
```

MongoDB will:

- NOT check all 100 documents
- Directly jump to the location of **"Arun"** using the index
→ So the query becomes **very fast** ⚡



Check Existing Indexes

```
db.Users.getIndexes()
```

Example Output:

id

name_1

Clusters

What is a Cluster

A cluster is a group of servers that work together to store your MongoDB data.

- A single cluster contains:
 - many databases
 - inside the databases → collections (tables)
 - inside collections → documents (rows)

So:

Cluster → Databases → Collections → Documents



Localhost & Clusters



Localhost → usually one cluster

When you install MongoDB locally (127.0.0.1), it usually runs one server + one cluster.



But if localhost runs multiple MongoDB servers

Then it becomes :

- ✓ group of servers →
 - ✓ working together →
 - ✓ forming a sharded / replicated cluster
-

Ultra Simple One-Line Definition

👉 Cluster is a group of servers that store MongoDB data together (databases + collections + documents).



REVISE-X